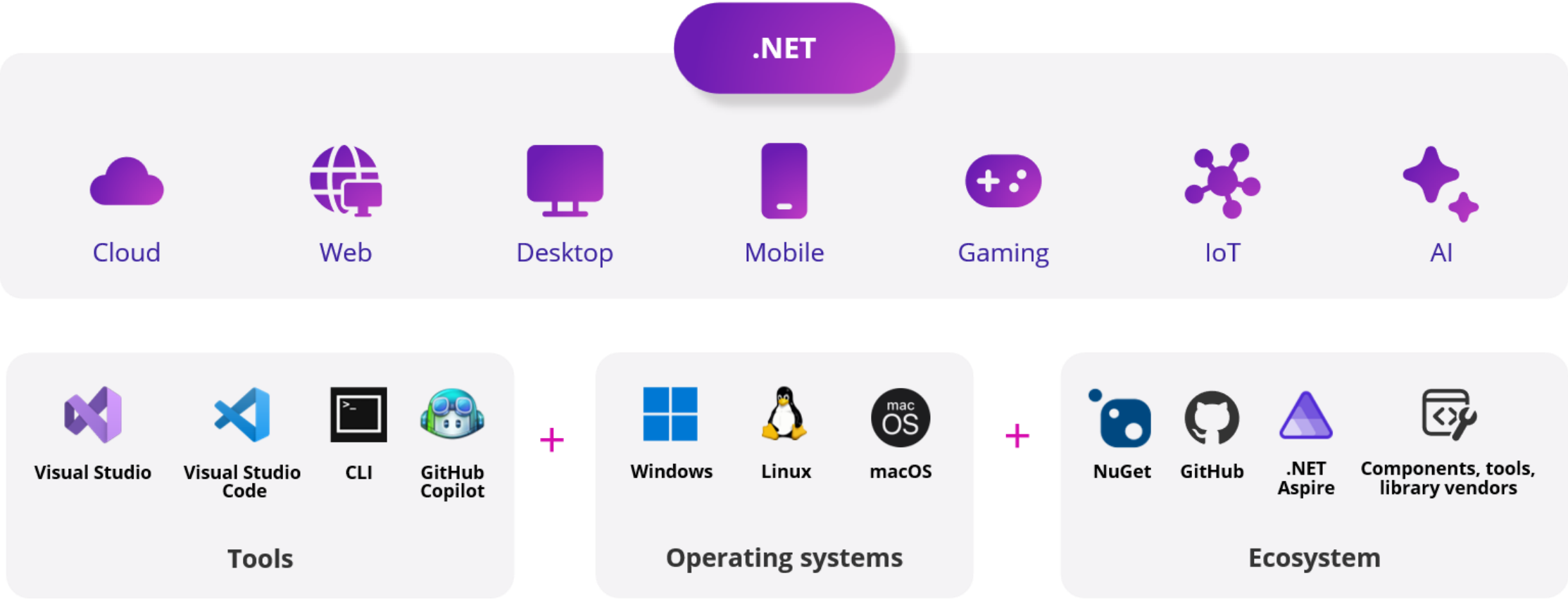
Langage C# - Les fondamentaux

Animé par Sylvain Labasse

INTRODUCTION



EN FIN DE MODULE, VOUS SAUREZ…

Ecrire un programme simple en C#

Regrouper des instructions dans des méthodes

Structurer une classe C#

Manipuler des tableaux et collections génériques

Pre-Requis

Public

SN2 - Développeur d’applications

Nécessaire

Un langage dérivé du C : C/C++, java, javascript ou PHP

Bases d’algorithmie

Notion de classe

DEROULEMENT

Support – 360 Learning

Copie des slides

Notes de cours

Ateliers

Evaluation

Ateliers sur 4

0	1	2	3	4
Non remis	Hors sujet	<	=	>
		attentes		

L'ENVIRONNEMENT

Matériel

PC + Connexion Internet

Logiciel

Visual Studio .Net communautaire 2022

Langage C# - Fondements

Plateforme .Net

- Enjeu
- Environnement d’exécution contrôlée
- Premier programme

Bases .Net et C#

- Plateforme .Net
- Variables
- Instructions
- Fonctions

Langage C# - Fondements

Classes

- Concepts
- Syntaxes élémentaires
- Méthodes, paramètres et propriétés
- Généralisation/Spécialisation

Collections

- Tableaux « primitifs »
- List<>
- Dictionary<>

Langage C# - Fondements

Blazor

- Fondements
- Binding
- Composants

BASES .NET ET C#

Objectifs

Rôle d'un environnement d'exécution contrôlée

Prise en main des outils et du C#

Manipulation des données

Traitements conditionnels et répétitifs

Regroupement d'actions sous forme de fonctions

Bases .Net et C#

- ➔ Plateforme .Net
- Variables
- Instructions
- Fonctions
- Synthèse

.NET

Genèse ~2000

Langage MS : Visual C++ = prog système / Visual Basic = Gestion
Age d'or Java EE (Sun, Oracle) => Microsoft fait son Java
C# évolue souvent avec .Net (1 par an)

Architecture

Multiplateformes grâce au CLR (env. d'exécution contrôlée)
Mémoire automatisée – Ramasse-miette (Garbage coll.)
Gestion des droits d'accès et dépendances
ASP.Net (web), MAUI (UI, Mobile)

C#

Langages

Aussi F#, VB.Net, …

Compilation en MIL puis Compilation « à la volée »

Produit un assemblage (dll ou exe sous windows)

Premier programme C#

Commentaires // ou /* */, régions #region … #endregion

```
// dotnet new console -n Bonjour
Console.WriteLine("Bonjour tout le monde !");
```

Bases .Net et C#

- ✓ Plateforme .Net
- ➔ Variables
- Instructions
- Fonctions
- Synthèse

Variables

Déclaration

Entier : byte, short, int, long / Réel : float, double, decimal

Autres : bool, char, string

int n ; ou var n = 10 ; (éventuellement const)

Nullable

int? n = null ;

if(n.HasValue) { int x = n.Value ; … }

Constantes

Nombres

0b0100_1010, 0112, 74, 0x4a

Typage : 2.5f, 2.5m, 123l, 123ul

Chaînes

"Bonjour \n Au revoir"

Multilignes : @"c:\news.txt" ou """ … """

$"La moyenne est {somme/qte} "

OPERATEURS

Opérations

(voir C/C++)

??, ??=

Transtypage

if(o is int n && n > 3) …

Conversions : var z = (Type)n; ou var z = n as Type ;

TYPES PERSONNALISES

Enumérations

Déclaration : enum Carte { Pique, Cœur, Carreau, Trefle }

Usage : Carte c = Carte.Pique ;

Tuples

Déclaration : (int Qte, decimal Prix) ligne;

Usage : ligne.Qte = 3; ou ligne = (3, 4.50m)

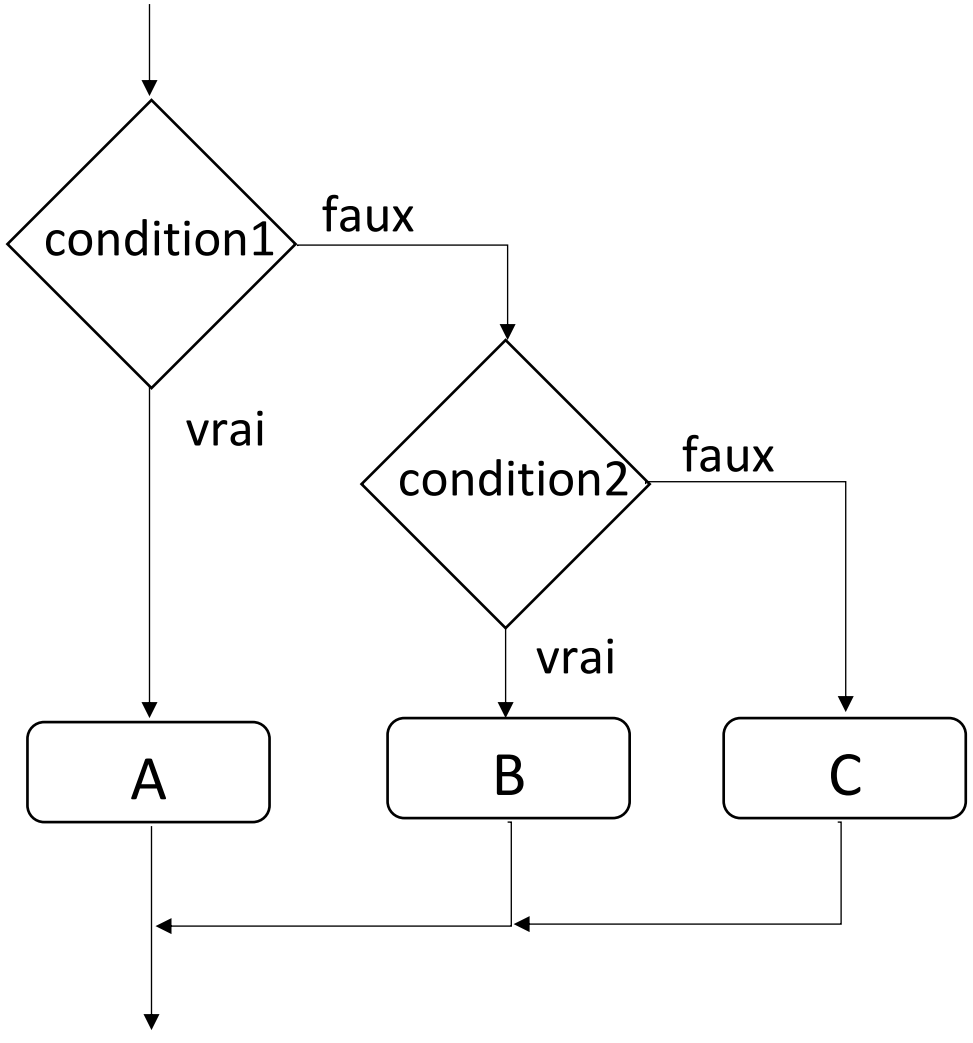
Déconstruction : (var q, var p) = ligne ;

Bases .Net et C#

- ✓ Plateforme .Net
- ✓ Variables
- ➔ Instructions
- Fonctions
- Synthèse

IF

```
if (condition1)
    { /* instructions A */ }
else if(condition2)
    { /* instructions B */ }
else
    { /* instructions C */ }
```



Switch

switch(entier) {

```
    case 1 : case 2 : … ; break;
    case 3 : … ; break;
    default : … ; break;
}
```

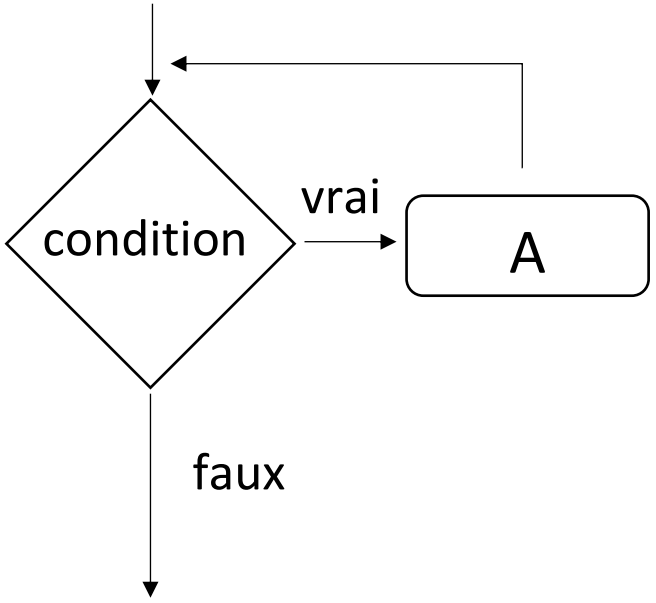
Ou expression = entier switch { 1 => …; 2 => … ; _ => … ; }

Equivalence

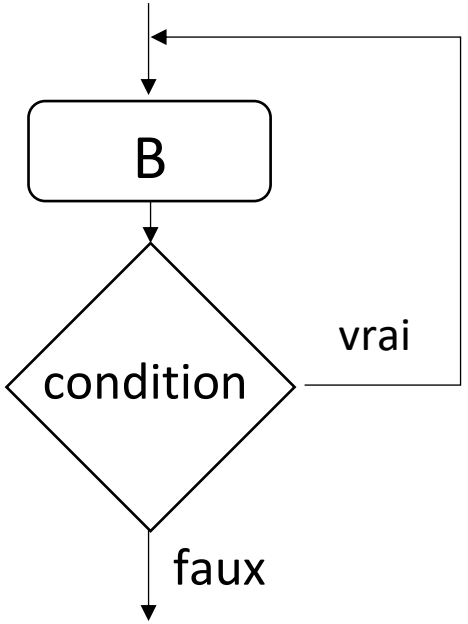
```
if(entier==1 || entier==2) { … } else if(entier==3) { … } else { … }
```

Boucle while / do...while

```
while(condition)
{
    // instruction A
}
```



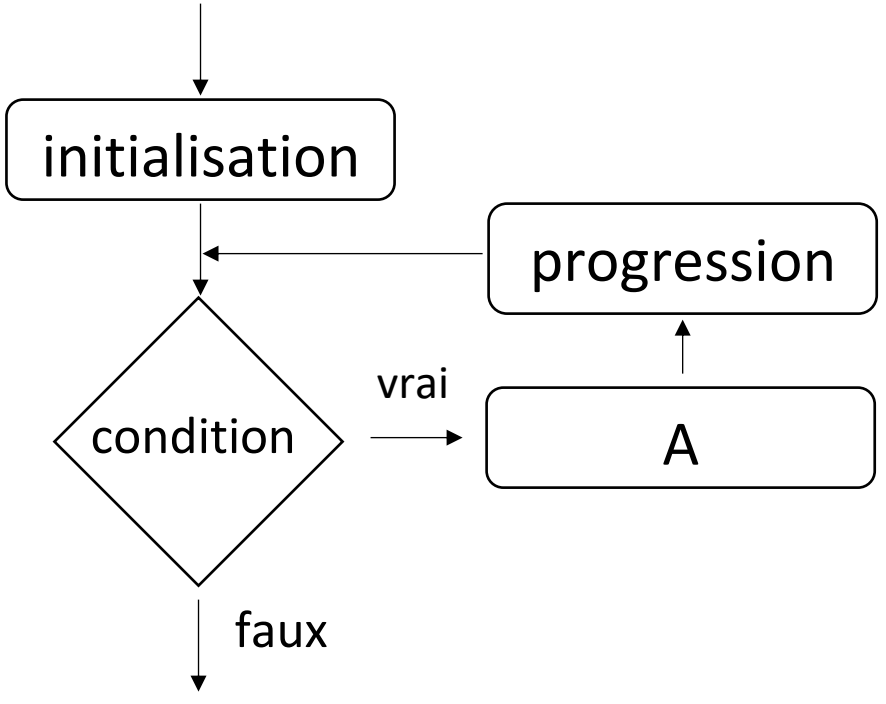

```
do
{
    // instruction B
}
while(condition);
```



FOR

```
for(initialisation; condition; progression)
{
    // instruction A
}

for(int i=0; i<n; i++)
{
    // instruction
}
```



Bases .Net et C#

- ✓ Plateforme .Net
- ✓ Variables
- ✓ Instructions
- ➔ Fonctions
- Synthèse

Fonctions

Définition

public void Methode(int param1, string param2) { … }

Syntaxe alternative {} : … => code;

Paramètres / Arguments

void f(int a, double b, int c)

Appel : f(x, y, 10) ou paramètres nommés f(c:10, a: x, b: y)

Bases .Net et C#

- ✓ Plateforme .Net
- ✓ Variables
- ✓ Instructions
- ✓ Fonctions
- ➔ Synthèse

RESUME

Rôle d’un environnement d’exécution contrôlée

Prise en main des outils et du C#

Manipulation des données

Traitements conditionnels et répétitifs

Regroupement d’actions sous forme de fonctions

CLASSES

Objectifs

Organisation d’une classe en C#

Encapsulation des données

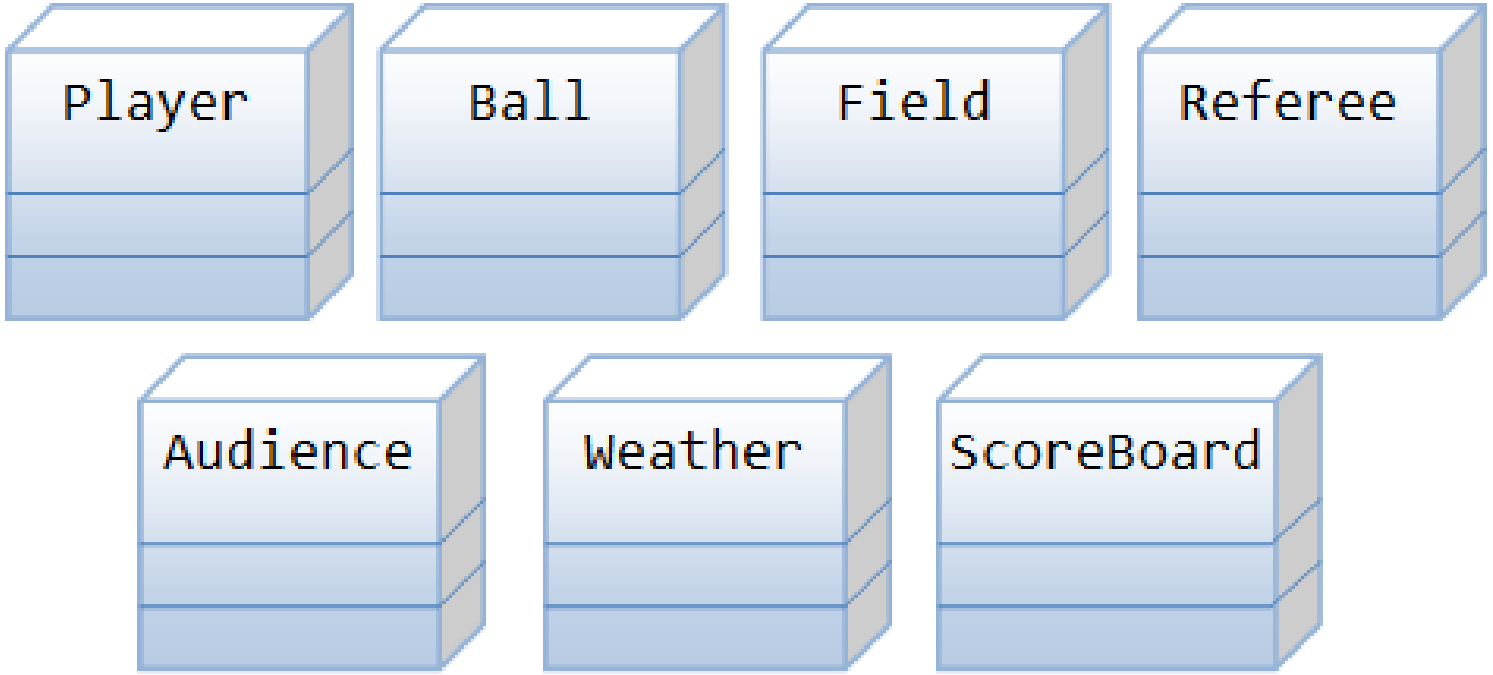
Accesseurs et fonctions de modification

Conception de classes SOLID

CLASSES

- ➔ Concepts
- Syntaxes élémentaires
- Méthodes, paramètres et propriétés
- Généralisation/Spécialisation
- Synthèse

Classes



Classes (Entities) in a Computer Soccer Game

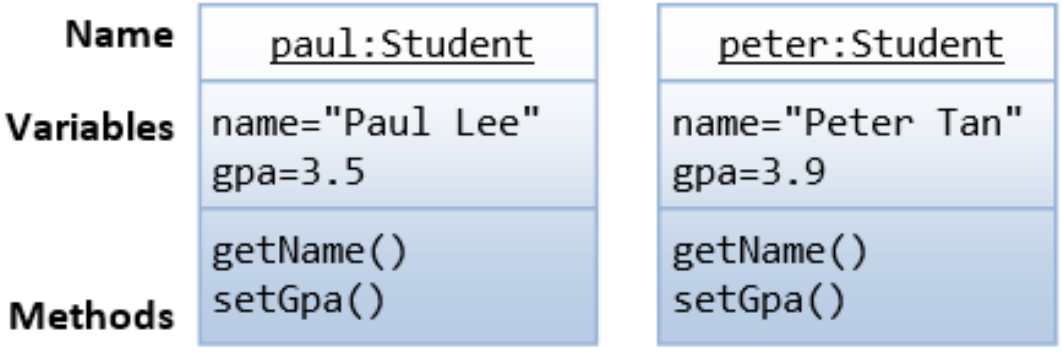
STRUCTURE

Name
(Identifier)
Variables
(Static attributes)
Methods
(Dynamic behaviors)

Student	Circle	SoccerPlayer	Car
name gpa	radius color	name number xLocation yLocation	plateNumber xLocation yLocation speed
getName() setGpa()	getRadius() getArea()	run() jump() kickBall()	move() park() accelerate()

Examples of classes

INSTANCES



Two instances - paul and peter - of the class Student

CLASSES

✓ Concepts

➔ Syntaxes élémentaires

Méthodes, paramètres et propriétés

Généralisation/Spécialisation

Synthèse

SYNTAXE ELEMENTAIRE

```
class Nom {
    private int champ ;
    public Nom(int c) {
        champ = c ;
    }
```

Instances

```
    Nom n = new Nom(10) ;
```

RECORD (C# 9)

Syntaxe positionnelle

record Personne (string Nom, int Age) { }

Méthodes implémentées : =, ==, !=, ToString, Deconstruct

class : Propriétés get/init / struct : get/set et def. constructor

Mutation non destructive

```
copie = original with {
    Prop1 = 3, Prop2 = ""
} ;
```

CLASSES

- ✓ Principe
- ✓ Syntaxes élémentaires
- ➔ Méthodes, paramètres et propriétés

  Généralisation/Spécialisation

  Synthèse

METHODES ET PARAMETRES

Méthodes

public void Methode(int param1, string param2) { … }

Syntaxe alternative {} : … => code;

Paramètres / Arguments

void f(ref int a, out double b, int c = 0)

Appel : f(ref x, out y, c : 10)

PROPRIETES

Syntaxe de base

```
int Nombre {
    get { return nb; }
    set { nb = value ; }
}
```

Propriété auto

```
int Nombre { get ; set ; } = 0 ;
```

EVENEMENTS

```
using System.Timers;

var timer = new System.Timers.Timer(1000);
var counter = 10;

timer.Elapsed += CountDown;
timer.Start();
while(counter > 0)
    Thread.Sleep(10);

1 référence
void CountDown(object? sender, ElapsedEventArgs e)
    => Console.WriteLine(--counter);
```

Exemple Timer.Elapsed

De type ElapsedEventHandler

Type indique les paramètres

+=/-= (dés)abonner à l'évènement

Utilité

Remplace les listeners Java ou design pattern Observer

Autre forme d'inversion de contrôle

Déclenchement synchrone : this.MonEvent?.Invoke(…);

LAMBDA (FONCTIONS ANONYMES JS)

Principe

Traitement transmis ou stocké pour usage ultérieur

Syntaxe : param1 => retour / (p1, p2) => { …; return …; }

Ex : timer.Elapsed += (s, e) => Console.WriteLine(--counter);

Types fréquents

Procédure : Action<int, float> a = (n, r) => Console.Write(…);

Fonction : Func<int, float, string> f = (n, r) => $"{n} / {r}";

Delegate : public delegate string MaFonction(int n, float r);

Evènements perso. : EventHandler<MonEventArgs> MonEvent;

CLASSES

- ✓ Principe
- ✓ Syntaxes élémentaires
- ✓ Méthodes, paramètres et propriétés
- ➔ Généralisation/Spécialisation
- Synthèse

GENERALISATION/SPECIALISATION

Héritage

Vue générale/Concept = Classe de base / abstraite

Sous-classes : Spécialisation

Polymorphisme

Action disponible « partout »

Code dépend de la sous-classe

Ex : getArea()

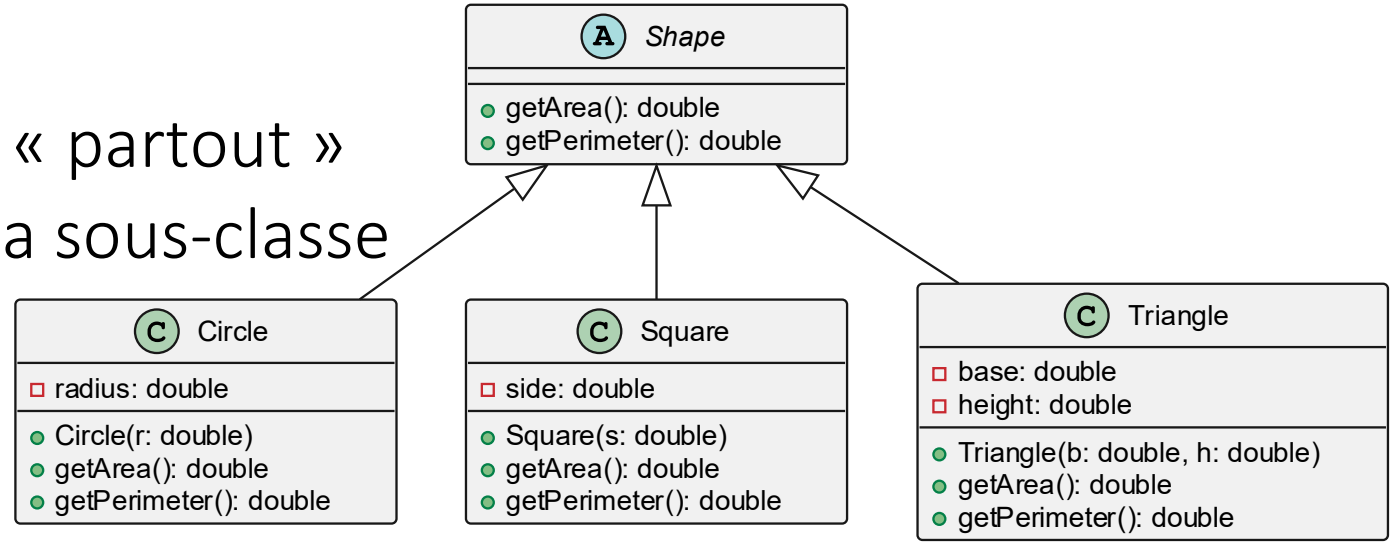


Figure 1 - src : https://mycodingdays.com/posts/object-oriented-modelling-part2/

INTERFACE = CONTRAT/CAPACITE

Contrat

Liste de méthodes/propriétés dispo.

Assimilable à un service min.

interface INomInterf {

void Signature(…) ;

Type Propriete { get ; }

// …

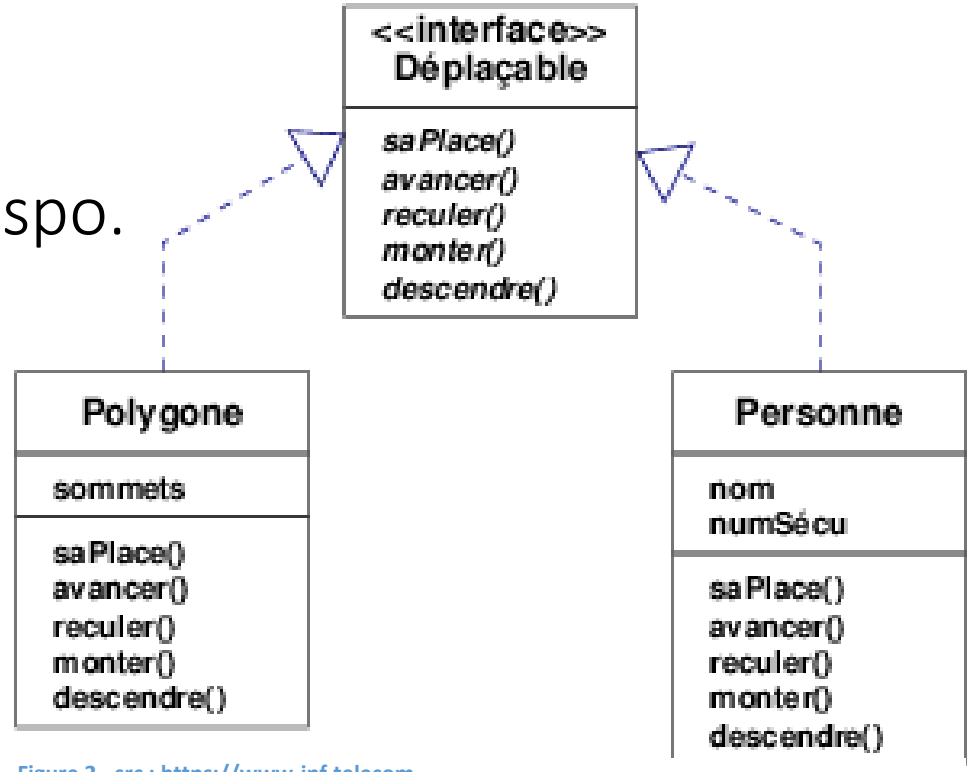


Figure 2 - src : https://www-inf.telecom-sudparis.eu/cours/CSC4002/EnLigne/Cours/CoursUML/5.18.34.html

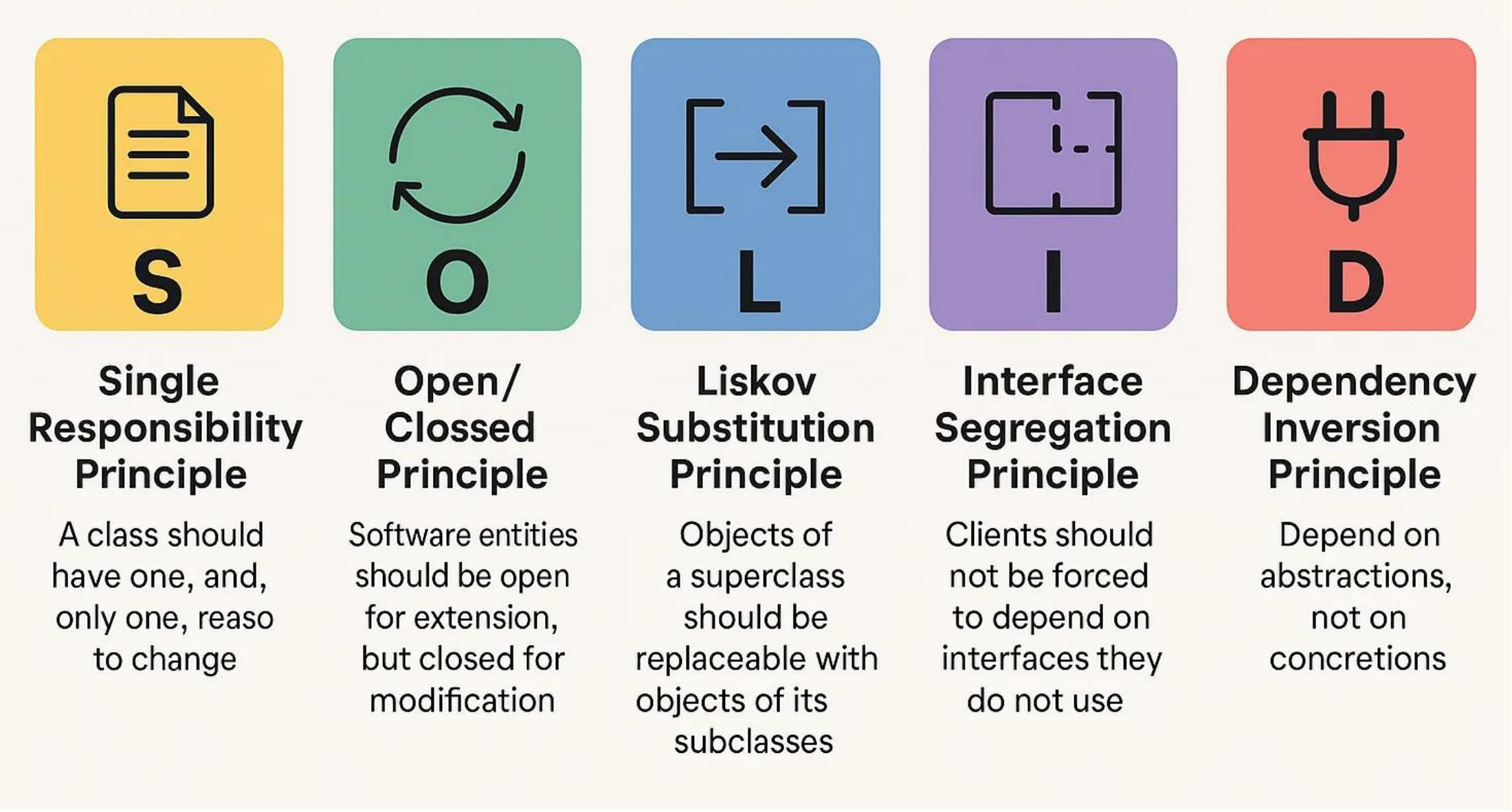


Figure 3 - src : https://techwayfit.com/blogs/design-principles/solid-principles/

Espaces de nom

Principe

Equivalent aux packages java

Au sein d'un même assemblage : accessibilité des « internal »

Using / global using

Utilisation des classes sans préfixe

Alias : using a.b.c.d as abcd; en cas de concurrence

CLASSES

- ✓ Principe
- ✓ Syntaxes élémentaires
- ✓ Méthodes, paramètres et propriétés
- ✓ Généralisation/Spécialisation
- ➔ Synthèse

RESUME

Organisation d’une classe en C#

Encapsulation des données

Accesseurs et fonctions de modification

Conception de classes SOLID

COLLECTIONS

Objectifs

Utilisation des tableaux de base

Avantages des List<> sur les tableaux

Structures permettant l'accès direct par clé

COLLECTIONS

➔ Tableaux « primitifs »

List<>

Dictionary<>

Synthèse

Tableaux « primitifs »

Unidimensionnel

```
int [] t = new int [10] ;
t = new int[]{ 25, 3, 12 };
t[0] = 6 ;
```

Multidimensionnel

```
int [,] u;
int [][] v ;
```

Tableaux « primitifs »

foreach

```
foreach(var n in t) {
    // n non modifiable
}
```

Opérations impossibles

Redimensionnement

Insertion, suppression avec décalage

COLLECTIONS

- ✓ Tableaux « primitifs »
- ➔ List<>
- Dictionary<>
- Synthèse

LIST<>

Usage

```
var l = new List<int>() ;
```

Méthodes utiles

Add, Remove, Clear

[], Count

COLLECTIONS

- ✓ Tableaux « primitifs »
- ✓ List<>
- ➔ Dictionary<>
- Synthèse

DICTIONARY<K, V>

Principe

Association clé/valeur

Accès quasi instantané par clé

Syntaxes

```
var d = new Dictionary<string, double>();
d["France"] = 3.4 ;
if(d.ContainsKey("France")) …
```

COLLECTIONS

- ✓ Tableaux « primitifs »
- ✓ List<>
- ✓ Dictionary<>
- ➔ Synthèse

RESUME

Utilisation des tableaux de base

Avantages des List<> sur les tableaux

Structures permettant l'accès direct par clé

BLAZOR

Objectifs

Repères de base dans les projets Blazor

Composants paramétrables et réutilisables

Mise à disposition des données

Traitement d’interactions utilisateur

BLAZOR

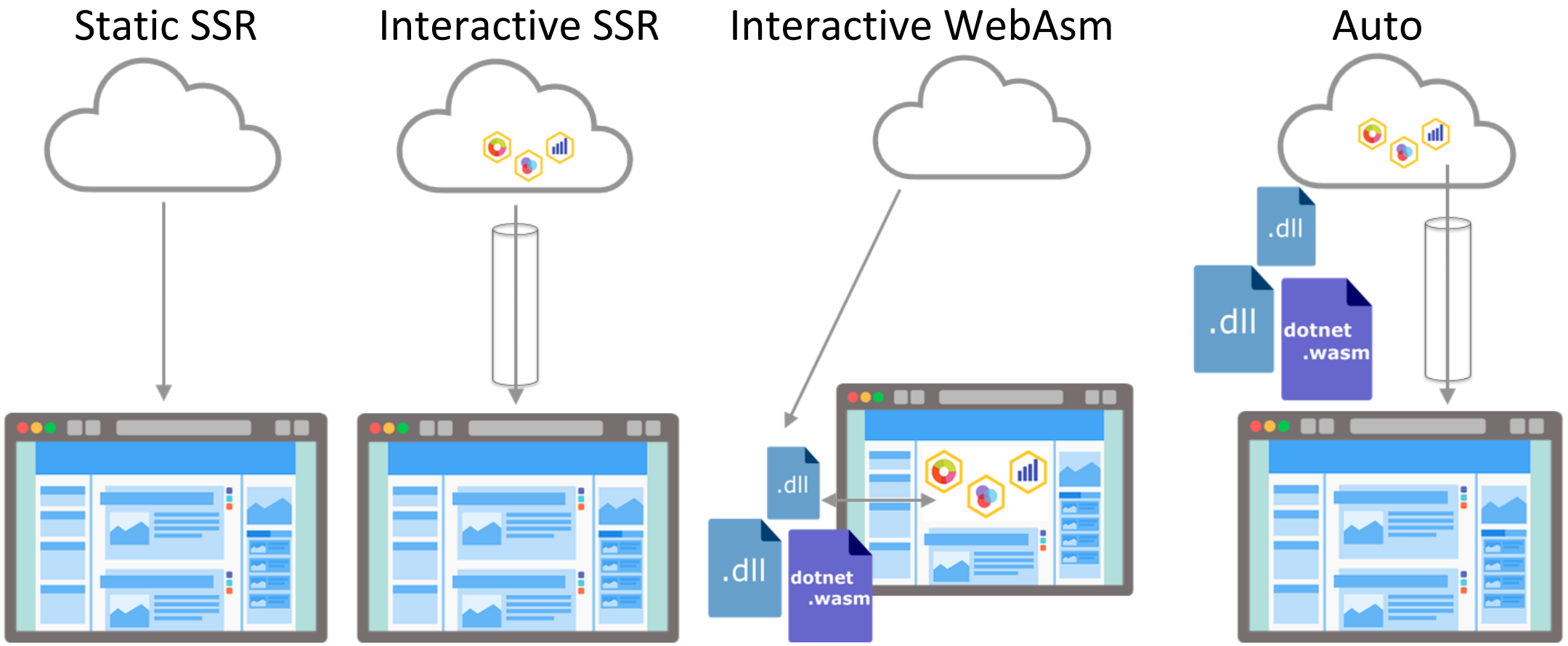
➔ Fondements

Binding

Composants

Synthèse

Modes de rendu



INITIATION A BLAZOR / PARAMETRAGES

Arborescence

Assets : wwwroot

_Imports.razor, App.razor

Program.cs

Pages, Shared : .razor

@page "/url"

@code { … } ou page.razor.cs partielle

HEBERGEMENT DE L'APPLICATION

Statique

N'importe quel CDN

Ex : Déploiement sur GitHub Page via GitHub Actions[1]

Dynamique

Nouveau projet / API ASP.Net

Ajout NuGet Microsoft.AspNetCore.Components.WebAssembly.Server

app.UseBlazorFrameworkFiles(), app.UseStaticFiles() ;

app.MapFallbackToFile("index.html") ;

[1] https://labasse.github.com/BlazorApp0 visible sur https://labasse.github.io/BlazorApp0/

BLAZOR

✓ Fondements

➔ Binding

Composants

Synthèse

Data Binding

Données

Utilisation de @champ

@code { private Objet champ=new (…); }

Attribut cond. `<input type="checkbox" checked="@IsChecked"/>`

Provocation d'un rendu : StateHasChanged() ;

Binding

<input @bind="date" @bind:format="dd/MM/yyyy"/>

@bind:event="oninput"

.Net 7 @bind:after="Meth.", :set="Meth.", :get="Membre"

LIAISONS ET EVENEMENTS

@onxxxx=Méthode ou lambda

EventArgs[2] : General, Clipboard, Drag, Error, Input, Media, Focus, Keyboard, Mouse (**onclick**, …), Progress, Touch

Contrôle

@onxxxx:preventDefault : Comportement par défaut inhibé

@onxxxx:stopPropagation : Pas propagé aux parents

OnAfterRender : DOM disponible

[2] https://docs.microsoft.com/en-us/aspnet/core/blazor/components/event-handling?view=aspnetcore-5.0#event-arguments-1

DEBOGAGE

Serveur ou client

Point d'arrêt sur le code CS ou razor

VS 2022/2026 : Live Reload

Points d'arrêt difficiles[3]

Exceptions non gérées

Démarrage côté client : Program.cs, OnInitialize

Non local : WSL ou codespaces

[3] https://learn.microsoft.com/en-us/aspnet/core/blazor/debug

BLAZOR

- ✓ Fondements
- ✓ Binding
- ➔ Composants
- Synthèse

Composants

Shared

.razor sans @page

.razor.css ajouté à <link href="{assemblage}.styles.css" rel="stylesheet">

Directives

@using …

@preservewhitespace true

Paramètres

[Parameter]

Passage : <Composant param1="@val" param2="@o.f()">

.Net6 : [EditorRequired]

Ne pas modifier directement sur le paramètre (le copier avant)

Autres paramètres

```
[Parameter(CaptureUnmatchedValues=true)]
public IDictionary<string, object> Misc { get ; set ; }
```

CYCLE DE VIE

Principaux

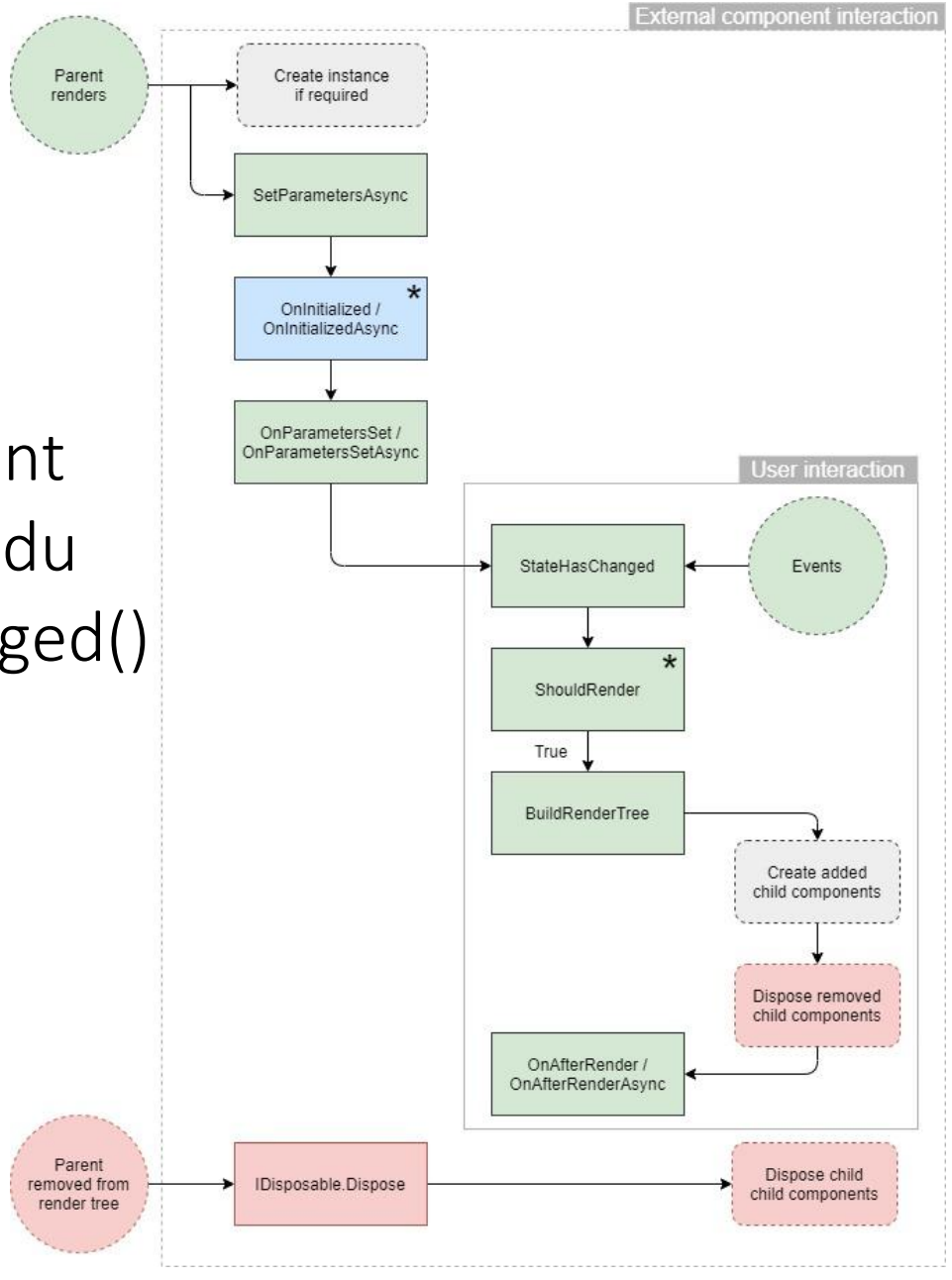
OnInitialized : 1er rendu seulement

ShouldRender : jamais au 1er rendu

Appel explicite de StateHasChanged()

Secondaire

OnParameterSet



BLAZOR

- ✓ Fondements
- ✓ Binding
- ✓ Composants
- ➔ Synthèse

RESUME

Repères de base dans les projets Blazor

Composants paramétrables et réutilisables

Mise à disposition des données

Traitement d’interactions utilisateur

BILAN

MAINTENANT, VOUS SAVEZ…

Ecrire un programme simple en C#

Regrouper des instructions dans des méthodes

Structurer une classe C#

Manipuler des tableaux et collections génériques

ET APRES…

DEVE424 – Langage C#, Tests unitaires

DEVE427 – Développement d'applications & TMA

PROE430 – Atelier UML, diagrammes